

Geometric–Semantic Fusion for Robotic Affordance Perception

Project Checkpoint Report

Francesco Dorati
Computer Vision – UPC

May 2026

1 Project overview

The project develops a multi-task perception pipeline for **robotic affordance extraction**: given a single RGB image of an arbitrary tool, the network predicts (i) a pixel-perfect 2D mask of the graspable region and (ii) a dense surface-normal map for the gripper approach direction. Combined with a depth reading at the mask centroid, these two outputs yield a full 6-DoF approach pose (X, Y, Z, N_x, N_y, N_z) that a motion planner can execute without any prior 3D CAD model of the object.

The deliverable for the course is the model, the training and evaluation code, and a quantitative report on instance-split generalization on the UMD Part Affordance Dataset, plus a qualitative test on in-the-wild captures from a different depth camera.

2 System scheme

The pipeline is an encoder–decoder with two prediction heads (Fig. 1).

1. **Encoder.** A frozen DINOv2 ViT-Small ingests a 448×448 RGB image. Instead of using only the final transformer block, features are tapped from *four* intermediate layers ($\ell \in \{2, 5, 8, 11\}$), each of shape $[B, 384, 32, 32]$. Earlier blocks retain local detail before global self-attention diffuses it – the same insight used in DPT.
2. **Decoder.** The four ViT scales are 1×1 -projected, concatenated, and fused at 32×32 . The decoder then upsamples progressively to 448×448 ($32 \rightarrow 56 \rightarrow 112 \rightarrow 224 \rightarrow 448$). At each stage, features from a small trainable RGB stem (channels 32/64/96/128 at 448/224/112/56) are concatenated as skip connections; these supply the high-frequency detail the ViT cannot reconstruct alone.
3. **Heads.** A mask head outputs raw logits (paired with BCEWithLogitsLoss + soft Dice for numerical stability and class-imbalance robustness). A separate normal head has its own Conv-BN-ReLU refinement and produces an L2-normalized 3-channel unit-vector field.

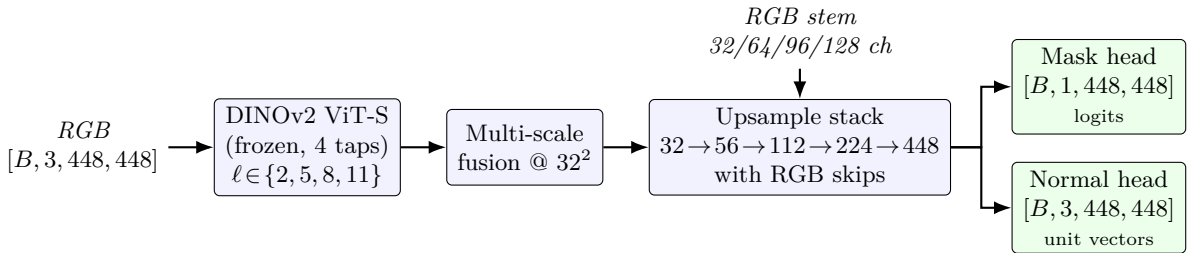


Figure 1: High-level dataflow: frozen ViT encoder → multi-scale fusion → progressive upsampling with trainable RGB skips → two task heads (mask logits and unit-normal vectors). Full diagram in `docs/architecture_diagram.svg`.

3 Components and partial implementation

The full code skeleton is in place and the forward pass runs end-to-end. Table 1 summarizes each component, where it lives, and its current state. The full loss is

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{DiceBCE}}(\hat{m}, m) + w_n \mathcal{L}_{\text{cos}}^{\text{masked}}(\hat{n}, n, m) + w_s \mathcal{L}_{\text{smooth}}(\hat{n}, I), \quad w_n = 5.0, \quad w_s = 0.5.$$

Component	Location	Status
Dataset & labels	<code>utils/dataset.py</code>	Done. 28,843 UMD samples loaded; grasp classes (1, 7) merged into a binary mask; instance-level 80/20 split.
On-the-fly normals	<code>utils/geometry.py</code>	Done. Depth back-projection with per-sensor intrinsics; principal point shifted after the 448×448 center crop.
Joint augmentations	<code>utils/augmentations.py</code>	Done. Rotation, scale, h-flip, photometric jitter, noise, random erasing – applied <i>consistently</i> to RGB / mask / normals (normal vectors are rotated and x-flipped to remain physically correct).
DINOv2 backbone (4-tap)	<code>models/backbone.py</code>	Done. Frozen, returns 4 ViT features.
Multi-task decoder	<code>models/decoder.py</code>	Done. Multi-scale fusion + RGB skip stack + dual heads.
Loss stack	<code>utils/losses.py</code>	Done. DiceBCELoss for mask, masked cosine for normals, edge-aware ($\exp(- \nabla \text{RGB})$) smoothness.
Training loop	<code>scripts/train.py</code>	Done. AdamW, JSONL logger, <code>best.pth/last.pth</code> , <code>-resume</code> .
Evaluation & reporting	<code>scripts/evaluate.py</code> , <code>visualize.py</code>	Done. IoU sweep @ 0.3–0.7, mean angular error, NYUv2 normal bins (11.25°/22.5°/30°), per-tool breakdown, training-curve plots.
Full training run	on Colab / remote GPU	Pending.
In-the-wild qualitative set	<code>data/custom_test_set/</code>	Pending.
RGB-D fallback variant	decoder branch	Deferred to iteration 2 (see §5).

Table 1: Implementation status by component at the checkpoint.

4 Initial findings

Baseline ablation drove the current design. An earlier single-scale ViT decoder was implemented first (preserved under `archive/v1/`). It revealed four problems that motivated the current architecture; each one became a concrete design change (Table 2). The v1 code is kept frozen so any number attached to it remains reproducible for the final A/B in the report. **Data pipeline sanity check.** A forward pass through

Problem in baseline	Fix in current arch.	Expected effect
Single 32×32 ViT tap; high-frequency detail had to be hallucinated	Multi-scale ViT fusion + trainable RGB skip stem	Sharper affordance boundaries, especially on thin handles
<code>sigmoid</code> inside decoder + <code>nn.BCELoss</code> (unstable)	Raw logits + <code>BCEWithLogitsLoss</code> + soft Dice	Stable gradients; better recall under heavy class imbalance
Hardcoded Kinect intrinsics; principal point not shifted after crop	Per-sensor <code>CAMERA_INTRINSICS</code> , c_x, c_y shifted to the crop	Geometrically correct ground-truth normals; transferable to other cameras
No augmentation; UMD always centers its tools	Joint geometric + photometric pipeline with normal-consistent transforms	Better generalization to off-center / rotated / in-the-wild inputs

Table 2: Findings from the v1 baseline that shaped the current architecture.

`UMDAffordanceDataset` returns 28,843 valid samples and tensors with the expected shapes (`rgb`: [3, 448, 448], `mask`: [1, 448, 448], `normals`: [3, 448, 448]). Visual inspection of the dynamically generated normals on `knife_01`, `mug_01`, and `saw_02` matches the expected geometry (handle planar, blade gradient, mug curvature).

Forward pass works. On a single batch ($B=8$) the backbone returns four [8, 384, 32, 32] feature maps and the decoder produces [8, 1, 448, 448] logits and [8, 3, 448, 448] unit-normal predictions. Output shapes and value ranges are correct.

5 Next steps toward the final report

1. **Full training on Colab.** Run the primary `-epochs 25` schedule with augmentation, plus an ablation run with `-no_augment` for the headline comparison and an A/B against the archived v1 baseline.
2. **Quantitative evaluation.** Report IoU at thresholds $\{0.3, \dots, 0.7\}$, mean angular error in degrees, and NYUv2 normal bins on the held-out tool instances, with a per-tool breakdown to identify failure categories (reflective, thin, dark objects).
3. **Qualitative sim-to-real.** Run inference on the custom office captures with the appropriate camera intrinsics, and produce the side-by-side prediction grids (`scripts/visualize.py -n_samples`).
4. **If time allows.** RGB-D variant (parallel depth encoder fusing into the decoder at 56^2 and 112^2) and a partial DINOv2 unfreeze of the last 2–4 blocks at a $10\times$ lower learning rate.

Repository. Code is version-controlled at https://github.com/francesco-dorati/robotic_affordance_project. The architecture is documented in `docs/ARCHITECTURE.md` and `docs/architecture_diagram.svg`; the evolution from the baseline is logged in `docs/CHANGELOG.md`.